CarbonIQ

Household Energy & Carbon Monitoring Prediction & Reduction Application

College AIML Mini-Project

Comprehensive Technical Documentation

Generated: October 2026

Version: 1.0 (MVP)

Table of Contents

1. Project Overview 3
2. System Architecture 4
3. Backend (FastAPI) 5
4. Frontend (React + TypeScript) 7
5. Machine Learning Model 9
6. Emission Factor & Carbon Calculation 11
7. API Reference 12
8. Implemented Features (MVP) 13
9. Not Implemented & Limitations 14
10. Project Structure 15
11. Quick Start Guide 16
12. Development Commands 17
13. Future Enhancements 18

1. Project Overview

1.1 What Is CarbonIQ?

CarbonIQ is a household energy and carbon monitoring, prediction, and reduction application developed as a college AI/ML mini-project. It predicts next-month electricity consumption for a household using a Seasonal Gradient Boosting model trained on Pune household survey data (January 2018 - February 2020). CO2 emissions are calculated deterministically from predicted kWh using the Central Electricity Authority (CEA) emission factor (0.675 kg CO2/kWh, database version 22.0, reporting period FY 2025-26).

1.2 Problem Statement

Households lack accessible tools to forecast their electricity consumption and understand the associated carbon footprint. Existing solutions either require smart meters, complex installations, or provide only historical data without predictive capability. CarbonIQ addresses this by providing a lightweight, survey-data-driven forecasting tool that runs entirely in the browser with a local backend.

1.3 Key Objectives

- Forecast next-month household electricity consumption (kWh) using only two prior months of data
- Calculate deterministic CO2 emissions from predicted electricity using versioned CEA emission factor
- Provide a clean, interactive web dashboard for visualization and forecasting
- Maintain transparent documentation of model limitations and accuracy (R-squared approx 0.17)
- Deliver a reproducible ML pipeline from raw data to deployed API

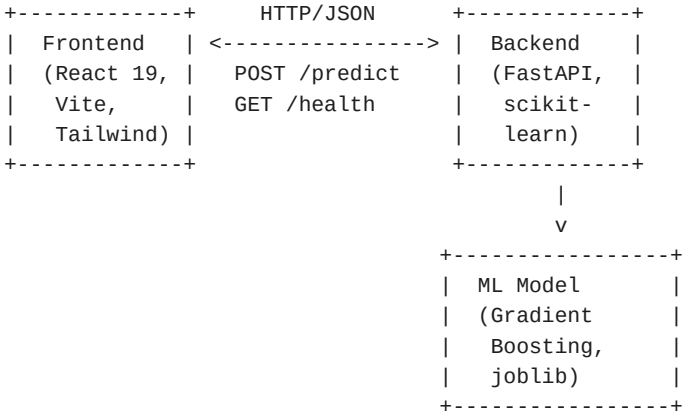
1.4 Scope & Boundaries

The MVP focuses exclusively on grid electricity. Other domains (LPG, Water, Food, Transport) are included in the frontend as placeholder pages with "Not Implemented" status due to insufficient verified emission factors or input data. This is by design and documented.

2. System Architecture

2.1 High-Level Architecture

CarbonIQ follows a client-server architecture with a React frontend communicating via HTTP/JSON to a FastAPI backend that serves the ML model.



2.2 Technology Stack

Frontend

- React 19 + TypeScript + Vite
- Tailwind CSS v4 (custom brand palette)
- Motion (Framer Motion v14) for animations
- Recharts for data visualization
- Axios for HTTP requests
- React Router v7 for client-side routing
- Three.js + React Three Fiber + @react-three/drei for 3D grid visualization
- Lucide React + Phosphor Icons for iconography

Backend

- FastAPI (async Python web framework)
- scikit-learn 1.6.1 for ML inference
- joblib for model serialization
- uvicorn ASGI server
- pandas for data preprocessing in API
- Pydantic for request/response validation

ML Model

- Algorithm: GradientBoostingRegressor (Seasonal variant)
- Features: electricity_t_minus_2, electricity_t_minus_1, target_month_num
- Target: Next-month electricity consumption (kWh)

- Training samples: 5,347 | Test samples: 1,434
- Model file: carboniq_seasonal_gb.joblib (~200 KB)
- Metadata: carboniq_model_metadata.json

3. Backend (FastAPI)

3.1 API Endpoints

GET /health

Health check endpoint to verify model loading and service status.

```
Response:
{
  "status": "ok",
  "model_loaded": true,
  "model_name": "Seasonal Gradient Boosting"
}
```

POST /predict

Main prediction endpoint. Accepts two prior months of electricity consumption and a target month number.

```
Request:
{
  "electricity_t_minus_2": 92,    // kWh, >= 0
  "electricity_t_minus_1": 114,   // kWh, >= 0
  "target_month_num": 3           // 1-12
}

Response:
{
  "predicted_electricity_kwh": 117.11,
  "predicted_co2_kg": 79.049,
  "emission_factor_kg_co2_per_kwh": 0.675,
  "factor_source": "Central Electricity Authority (CEA)",
  "factor_version": "22.0",
  "reporting_period": "FY 2025-26",
  "gas": "CO2",
  "model_name": "Seasonal Gradient Boosting",
  "model_metrics": {
    "MAE_kWh": 32.1234,
    "RMSE_kWh": 186.3864,
    "R2": 0.1689
  },
  "limitations": [...]
}
```

3.2 Request Validation

- electricity_t_minus_2: float, >= 0, no inf/NaN
- electricity_t_minus_1: float, >= 0, no inf/NaN
- target_month_num: integer, 1-12 inclusive
- All fields required

3.3 CORS Configuration

Configured for local development origins:

- http://localhost:5173
- http://127.0.0.1:5173
- http://localhost:4173 (Vite preview)
- http://127.0.0.1:4173

3.4 Model Loading

On startup, the backend loads two artifacts from the backend/ directory:
1. carboniq_seasonal_gb.joblib - the trained GradientBoostingRegressor model
2. carboniq_model_metadata.json - metadata including features, metrics, emission factor, and limitations

The emission factor (0.675 kg CO2/kWh) is read from metadata at runtime, not hardcoded, allowing updates without code changes.

4. Frontend (React + TypeScript)

4.1 Pages & Routes

Route	Page	Description
/	Dashboard	KPI cards, domain overview, latest forecast summary, quick actions
/electricity	Electricity	Forecast form, 3D power grid visualization, seasonal charts, emission factor info
/forecast	Forecast	Dedicated prediction form, localStorage history (20 entries), model details
/lpg	LPG	Placeholder - planned CO2 calculator
/water	Water	Placeholder - planned CO2 calculator
/food	Food	Placeholder - planned CO2 calculator
/transport	Transport	Placeholder - vehicle ownership display only
/todo-not-todo	Todo/Not-Todo	Static rule-based energy saving tips
/settings	Settings	Emission factor override (localStorage), units toggle, model metadata

4.2 Key Components

- **UI Primitives:** Button, Card, Input, Select, Badge - consistent styling with Tailwind
- **Layout:** Sidebar, Header, PageContainer - responsive layout with navigation
- **Animations:** StaggerContainer/StaggerItem, ElevatedCard, MotionButton - Framer Motion powered
- **Charts:** Recharts AreaChart (seasonal trends), BarChart (appliance breakdown)
- **3D Visualization:** PowerGridIsland - Three.js animated grid with workload-responsive turbines/steam
- **Hooks:** useForecast (prediction state), useHealth (backend polling every 30s)
- **API Client:** Axios instance with base URL, timeout, error interceptor

4.3 State Management

No global state library (Redux/Zustand) used. State is managed via:
- React useState/useReducer for component-local state
- Custom hooks (useForecast, useHealth) for shared async logic
- localStorage for forecast history persistence (Forecast page)
- localStorage for user preferences (Settings page)
- URL/route state via React Router

4.4 Styling & Theming

Tailwind CSS v4 with custom brand palette:

Token	Hex	Usage
brand-950	#051F20	Primary dark (headers, text)
brand-900	#0B2B26	Dark variant
brand-800	#163832	Card backgrounds

brand-700	#235347	Primary brand (buttons, accents)
brand-300	#8EB69B	Mint green (secondary, charts)
brand-100	#DAF1DE	Light mint (backgrounds, highlights)

5. Machine Learning Model

5.1 Model Specification

Property	Value
Algorithm	GradientBoostingRegressor (Seasonal)
Library	scikit-learn 1.6.1
Features	electricity_t_minus_2, electricity_t_minus_1, target_month_num
Target	electricity_target (kWh/month)
Training Samples	5,347
Test Samples	1,434 (Nov 2019 - Feb 2020)
Training Cutoff	Before November 2019 (chronological split)
Model File	carboniq_seasonal_gb.joblib
Serialization	joblib.dump / joblib.load

5.2 Hyperparameters

Parameter	Value
n_estimators	200
learning_rate	0.05
max_depth	3
subsample	1.0
min_samples_split	2
min_samples_leaf	1
loss	squared_error
criterion	friedman_mse
alpha	0.9
random_state	42
validation_fraction	0.1
n_iter_no_change	None (early stopping disabled)

5.3 Evaluation Metrics

Metric	Value
MAE (Mean Absolute Error)	32.1234 kWh
RMSE (Root Mean Squared Error)	186.3864 kWh
R-squared (R2)	0.1689

5.4 Model Interpretation

The R-squared of 0.1689 indicates the model explains approximately 17% of the variance in next-month electricity consumption. This is modest predictive power. The MAE of 32.12 kWh means on average predictions are off by ~32 kWh (roughly 29% of typical ~110 kWh consumption). The high RMSE (186 kWh) relative to MAE suggests occasional large errors, likely on outlier households. The model is intended as a baseline seasonal forecaster, not a precision tool.

5.5 Training Pipeline Summary

- 1. Load Pune Household Survey CSV (1,872 households, 26 monthly columns)
- 2. Identify and chronologically sort monthly electricity columns (Jan 2018 - Feb 2020)
- 3. Construct genuine consecutive 3-month windows (t-2, t-1, target) - only rows with all 3 values present
- 4. Extract target_month_num (1-12) from target column name
- 5. Chronological split: train on data before Nov 2019, test on Nov 2019 - Feb 2020
- 6. Train Seasonal GradientBoostingRegressor with fixed hyperparameters
- 7. Evaluate on test set (MAE, RMSE, R2)
- 8. Serialize model (joblib) and metadata (JSON) for backend consumption

6. Emission Factor & Carbon Calculation

6.1 Emission Factor Details

Property	Value
Value	0.675 kg CO2/kWh
Source	Central Electricity Authority (CEA), Government of India
Database Version	22.0
Reporting Period	FY 2025-26
Gas	CO2 (carbon dioxide only, not CO2e)
Scope	Grid electricity only (Scope 2 indirect emissions)

6.2 Carbon Calculation Formula

```
predicted_co2_kg = round(max(0, predicted_electricity_kwh) * 0.675, 3)
```

The calculation is deterministic - no ML involved. The emission factor is applied directly to the model's electricity prediction. The factor is sourced from the backend at runtime via the /predict response, ensuring the frontend always uses the versioned factor from metadata.

6.3 Why Deterministic?

- Emission factors are policy/regulatory values, not learned from data
- CEA factor is updated annually; versioning ensures reproducibility
- Separates ML uncertainty (electricity prediction) from policy certainty (emission factor)
- Allows factor updates without retraining the model
- Transparent: users can verify calculation manually

6.4 Important Caveats

- Factor period (FY 2025-26) differs from ML training data period (2018-2020)
- Grid emission intensity changes over time (renewables penetration, fuel mix)
- Factor applies only to grid electricity, not LPG, transport, etc.
- No transmission/distribution losses included in this factor
- Regional variations within India not captured (single national average)

7. API Reference

7.1 Base URL

Development: http://127.0.0.1:8000

7.2 Common Headers

Header	Value
Content-Type	application/json
Accept	application/json

7.3 Error Responses

FastAPI returns standard HTTP status codes with JSON error details:

Code	Name	Description
400	Validation Error	Input validation failed (e.g., negative kWh, month > 12)
422	Unprocessable Entity	Pydantic validation error with field details
500	Internal Server Error	Model loading or prediction failure

7.4 Example curl Commands

```
# Health check
curl http://127.0.0.1:8000/health

# Prediction
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"electricity_t_minus_2": 92, "electricity_t_minus_1": 114, "target_month_num": 3}'
```

8. Implemented Features (MVP)

Feature	Description	Layer
Electricity Forecasting	POST /predict endpoint with t-2, t-1, target_month inputs	Backend + Frontend
Deterministic CO2 Calculation	0.675 kg CO2/kWh factor applied to predicted kWh	Backend + Frontend
Model Health Check	GET /health returns model load status and name	Backend + Frontend
Dashboard Page	KPI cards, domain status grid, quick actions, latest forecast summary	Frontend
Electricity Page	Forecast form, 3D PowerGridIsland, seasonal charts, emission factor c	Frontend
Forecast Page	Full prediction form, 20-entry localStorage history, model metrics displ	Frontend
LPG/Water/Food/Transport Page	Placeholder pages with status badges and descriptions	Frontend
Todo/Not-Todo Page	Static categorized energy-saving tips (client-side)	Frontend
Settings Page	Emission factor override, units toggle, model metadata display	Frontend
Responsive Design	Mobile-first Tailwind CSS, collapsible sidebar	Frontend
Animations	Staggered entrance, hover elevations, loading spinners, 3D canvas	Frontend
TypeScript Types	Full type coverage for API requests/responses, components	Frontend

9. Not Implemented & Limitations

9.1 By Design (Documented Gaps)

Domain	Status	Reason
LPG CO2	Not available	No verified emission calculation; survey data exists but no CEA-equivalent factor
Water CO2	Not available	No verified emission calculation implemented
Food CO2	Not available	Highly variable by diet/source; no generic factor without verification
Transport CO2	Not available	Vehicle ownership != distance/fuel use; survey lacks travel data
Historical Charts	Not available	No /history/:household_id endpoint; would need new API
Multi-household	Not available	API accepts raw inputs; no household lookup or management
Persistent History	Not available	No database; frontend uses localStorage only
User Authentication	Not available	Out of scope for MVP

9.2 Model Limitations (From Metadata)

- Trained on historical household electricity observations (2018-2020)
- Evaluation uses a historical test period (Nov 2019 - Feb 2020)
- Current-period forecasts need validation against newer data
- The emission factor is configurable and separate from the ML model
- R-squared approx 0.17 indicates modest explanatory power; predictions are estimates

9.3 Known Technical Debt

- No automated tests (backend or frontend)
- No CI/CD pipeline
- No Docker configuration
- No production deployment configs
- Single-household model (no household_id in API)
- No request rate limiting on /predict
- No structured logging or monitoring
- Frontend: no error boundaries, limited accessibility audit

10. Project Structure

```
.
+---- backend/                    # FastAPI + model artifacts
|     +---- app.py                 # FastAPI application
|     +---- carboniq_model_metadata.json
|     +---- carboniq_seasonal_gb.joblib
|     +---- requirements.txt
+---- frontend/                  # React + TypeScript + Vite
|     +---- public/
|     +---- src/
|     |     +---- api/              # Axios client, API calls
|     |     +---- components/
|     |     |     +---- ui/           # Button, Card, Input, Select, Badge
|     |     |     +---- layout/       # Sidebar, Header, PageContainer
|     |     |     +---- animations/   # PowerGridIsland (3D viz)
|     |     +---- hooks/            # useForecast, useHealth
|     |     +---- lib/animations.tsx # Motion animation utilities
|     |     +---- pages/            # Dashboard, Electricity, Forecast, etc.
|     |     +---- types/            # TypeScript interfaces
|     |     +---- App.tsx
|     |     +---- main.tsx
|     |     +---- routes.tsx
|     +---- package.json
|     +---- vite.config.ts
|     +---- tailwind.config.js
+---- notebooks/
|     +---- CarbonIQ_ML_Clean.ipynb    # Cleaned ML pipeline
+---- docs/
|     +---- FRONTEND_IMPLEMENTATION_PLAN.md
|     +---- CHANGE_REPORT.md
+---- scripts/
|     +---- check_nb.py
|     +---- count_original.py
+---- start.sh / start.bat / start.ps1  # Startup scripts
+---- README.md
+---- .gitignore
```

11. Quick Start Guide

11.1 Prerequisites

- Python 3.10+
- Node.js 18+ (includes npm)
- Git (optional)

11.2 Backend Setup

```
cd backend
python -m venv .venv
# Windows:
.venv\Scripts\activate
# Linux/macOS:
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app:app --reload
# Runs on http://127.0.0.1:8000
```

11.3 Frontend Setup

```
cd frontend
npm install
npm run dev
# Runs on http://localhost:5173
# CORS configured for this origin
```

11.4 One-Command Start (Windows)

Double-click start.bat or run in PowerShell:

```
.\start.ps1
```

Or use start.bat in Command Prompt. Both launch backend and frontend in separate windows.

11.5 API Verification

```
# Health check
curl http://127.0.0.1:8000/health

# Prediction example
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"electricity_t_minus_2": 92, "electricity_t_minus_1": 114, "target_month_num": 3}'
```

12. Development Commands

Task	Command
Frontend dev server	cd frontend && npm run dev
Frontend production build	cd frontend && npm run build
Frontend lint	cd frontend && npm run lint
Backend dev server	cd backend && uvicorn app:app --reload
Backend install deps	cd backend && pip install -r requirements.txt
Run ML notebook	jupyter notebook notebooks/CarbonIQ_ML_Clean.ipynb
Start all (PowerShell)	.\start.ps1
Start all (CMD)	start.bat
Start all (Git Bash/WSL)	./start.sh

13. Future Enhancements

13.1 High Priority

- Add walk-forward time-series cross-validation to notebook
- Implement /history/:household_id endpoint for historical charts
- Add SQLite/PostgreSQL for persistent forecast history & multi-household
- Dockerize both services with docker-compose
- Set up GitHub Actions CI (lint, test, build)
- Add pytest (backend) and Vitest (frontend) test suites

13.2 Medium Priority

- Feature engineering: cyclic month encoding (sin/cos), rolling statistics
- Try LightGBM/XGBoost for potentially better performance
- Add household metadata features (size, appliances, income) if available
- Implement LPG/Water calculators with IPCC/DEFRA default factors
- Add Prometheus metrics endpoint (/metrics) for monitoring
- Structured logging with correlation IDs
- Rate limiting on /predict endpoint

13.3 Low Priority / Nice to Have

- Dark mode support in frontend
- Export forecasts to CSV/PDF
- WebSocket live updates for real-time predictions
- Storybook for component documentation
- Model versioning & A/B testing framework
- SHAP feature importance in /predict response
- Automated drift detection on logged predictions